REVISION CHEAT SHEET:

SOLID / DESIGN

SOLID and Core Design Principles in C# and .NET

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

31.1 Why Design Principles Matter

31.2 SOLID Overview

31.3 Cohesion

31.4 Coupling

31.5 Design Goal

Part 1 — Single Responsibility Principle

31.6 What Is SRP?

31.7 SRP Is Not “One Method Per Class”

31.8 Bad SRP Example

31.9 Better SRP Design

31.10 SRP Fundraising Example

31.11 Constructor Dependency Smell

31.12 When Not to Split Further

Part 2 — Open/Closed Principle

31.13 What Is OCP?

31.14 Bad OCP Example

31.15 OCP with Strategy

31.16 OCP Does Not Mean “Never Modify Code”

31.17 Do Not Abstract Before You Know the Variation

Part 3 — Liskov Substitution Principle

31.18 What Is LSP?

31.19 LSP Is About Behavior

31.20 Contract Components

31.21 Preconditions

31.22 Postconditions

31.23 Exceptions and LSP

31.24 Classic Shape Problem

31.25 LSP Design Lesson

Part 4 — Interface Segregation Principle

31.26 What Is ISP?

31.27 Bad Interface

31.28 Better Segregated Interfaces

31.29 ISP Does Not Mean One-Method Interfaces Everywhere

31.30 ISP and Capabilities

Part 5 — Dependency Inversion Principle

31.31 What Is DIP?

31.32 Bad Dependency Direction

31.33 Inverted Dependency

31.34 Dependency Injection vs Dependency Inversion

Dependency Inversion

Dependency Injection

31.35 Inversion of Control

31.36 ASP.NET Core DI Example

31.37 Explicit Dependencies

Part 6 — Separation of Concerns

31.38 What Is Separation of Concerns?

31.39 Fundraising Layer Example

31.40 Bad Controller

31.41 Better Controller

Part 7 — Encapsulation and Information Hiding

31.42 Encapsulation

Better

31.43 Information Hiding

Part 8 — DRY

31.44 What Is DRY?

31.45 Bad Duplication

31.46 Better

31.47 DRY Does Not Mean “Never Repeat Code”

Part 9 — KISS

31.48 What Is KISS?

31.49 Simple Does Not Mean Primitive

Part 10 — YAGNI

31.50 What Is YAGNI?

31.51 YAGNI and OCP Balance

Part 11 — Composition over Inheritance

31.52 Why Prefer Composition Often?

31.53 Why Composition Is Flexible

31.54 When Inheritance Is Good

Part 12 — Law of Demeter

31.55 Law of Demeter

31.56 Why Deep Navigation Is Fragile

Part 13 — Tell, Don't Ask

31.57 Tell, Don't Ask

Part 14 — Principle of Least Knowledge and Explicit APIs

Part 15 — SOLID Applied Together

31.58 Improved Design

31.59 How Principles Apply

31.60 SOLID and Testing

Testable

31.61 Do Not Create Interfaces Only for Mocking

31.62 SOLID and Performance

31.63 SOLID and Memory

31.64 SOLID and Thread Safety

31.65 SOLID and Security

31.66 Service Locator Anti-Pattern

31.67 Static Dependency Anti-Pattern

31.68 God Class

31.69 Anemic Over-Splitting

31.70 Common Mistake — SRP Means One Method

31.71 Common Mistake — OCP Means Never Edit Existing Code

31.72 Common Mistake — LSP Is Just Type Compatibility

31.73 Common Mistake — ISP Means Tiny Interfaces Everywhere

31.74 Common Mistake — DIP Means “Use DI Container”

31.75 Common Mistake — DRY Everything

31.76 Common Mistake — Premature OCP

31.77 Common Mistake — Inheritance for Code Reuse

31.78 Common Mistake — Repository for Every Object Automatically

31.79 Common Mistake — Business Rules in Controller

31.80 Common Mistake — Business Rules in Infrastructure

31.81 Common Mistake — Too Many Interfaces

31.82 Best Practices

31.83 Practical Exercise 1 — SRP

31.84 Practical Exercise 2 — OCP

31.85 Practical Exercise 3 — LSP

31.86 Practical Exercise 4 — ISP

31.87 Practical Exercise 5 — DIP

31.88 Practical Exercise 6 — DRY

31.89 Practical Exercise 7 — Encapsulation

31.90 Practical Exercise 8 — Composition

31.91 Practical Exercise 9 — Explicit Dependencies

31.92 Practical Exercise 10 — KISS

31.93 Practical Exercise 11 — Multi-Tenant Security

31.94 Practical Exercise 12 — Refactor Large Service

Topic 31 — 25 Interview Questions

1. What is SOLID?
2. What is the Single Responsibility Principle?
3. How can constructor dependencies reveal an SRP problem?
4. What is the Open/Closed Principle?
5. Does OCP mean existing code must never change?
6. What is the Liskov Substitution Principle?
7. What kinds of behavior matter for LSP?
8. Why is the Rectangle/Square example associated with LSP?
9. What is Interface Segregation?
10. Does ISP require single-method interfaces?
11. What is Dependency Inversion?
12. What is the difference between DIP and dependency injection?
13. What is inversion of control?
14. Why is constructor injection usually preferred?
15. What is separation of concerns?
16. What is encapsulation?
17. What is DRY?
18. What is KISS?
19. What is YAGNI?
20. How do OCP and YAGNI coexist?
21. Why is composition often preferred over inheritance?
22. What is the Law of Demeter?
23. Can SOLID hurt a project?
24. How does SOLID improve testing?

25. How would you apply SOLID to the Fundraising Platform?
Topic 31 — Quick Revision
Topic 31 — Complete Fundraising Architecture Example
Controller
Domain Gift
Repository Abstraction
Application Service
Infrastructure Repository
Dependency Registration
Final Architecture Flow
Topics 30 + 31 — Combined Mental Model
Final Revision Flow

SOLID / DESIGN

31.1 Why Design Principles Matter

C# allows you to write this:

```C#
public sealed class GiftService
{
    public void ProcessGift(
        Gift gift)
    {
        ValidateGift(
            gift);

        SaveGift(
            gift);

        ChargePayment(
            gift);

        GenerateReceipt(
            gift);

        SendEmail(
            gift);

        WriteAuditLog(
            gift);

        UpdateCampaignTotals(
            gift);

        PublishMessage(
            gift);
    }
}
```

The compiler may be satisfied.

But design questions remain:

EXAMPLE / FLOW

```
How many reasons does GiftService have to change?

How do we test it?

How do we replace SMTP?

How do we change payment providers?

How do we add a new receipt mechanism?

What happens if auditing changes?
```

```
Can persistence change without rewriting business logic?

Can another developer understand the dependencies?
```

SOLID / DESIGN

SOLID and related design principles help answer these questions.

Microsoft's current .NET architecture guidance identifies SOLID as a foundational set of principles for internal application design and dependency decoupling.

31.2 SOLID Overview

SOLID means:

EXAMPLE / FLOW

```
S
Single Responsibility Principle

O
Open/Closed Principle

L
Liskov Substitution Principle

I
Interface Segregation Principle

D
Dependency Inversion Principle
```

A useful mental model:

EXAMPLE / FLOW

```
S
    →
Give a class one cohesive responsibility

O
    →
Create stable extension points

L
    →
Implementations must preserve the contract

I
    →
Do not force clients to depend on members they do not need

D
    →
High-level policy should depend on abstractions,
not low-level implementation details
```

31.3 Cohesion

Cohesion describes how closely related the responsibilities inside a component are.

High cohesion:

EXAMPLE / FLOW

```
GiftValidator
    ↓
Gift validation rules
```

SOLID / DESIGN

Low cohesion:

EXAMPLE / FLOW

```
GiftManager
    ↓
Validate
SQL
Email
PDF
Payment
CSV
Authentication
Caching
Logging
```

High cohesion usually improves:

EXAMPLE / FLOW

```
Understandability
Testability
Maintainability
Change isolation
```

31.4 Coupling

Coupling describes how strongly components depend on each other's details.

Tight coupling:

C#

```
public sealed class GiftService
{
    private readonly
        SqlGiftRepository
        _repository = new();

    private readonly
        SmtpReceiptSender
        _sender = new();
}
```

The service directly chooses:

EXAMPLE / FLOW

```
SQL implementation
SMTP implementation
Construction
Lifetime
```

Looser coupling:

C#

```
public sealed class GiftService
{
    private readonly
```

```
        IGiftRepository
        _repository;

    private readonly
        IReceiptSender
        _receiptSender;

    public GiftService(
        IGiftRepository repository,
        IReceiptSender receiptSender)
    {
        _repository = repository;

        _receiptSender = receiptSender;
    }
}
```

SOLID / DESIGN

Now the service knows:

EXAMPLE / FLOW

```
contracts
```

instead of:

EXAMPLE / FLOW

```
infrastructure implementations
```

31.5 Design Goal

A healthy design usually seeks:

EXAMPLE / FLOW

```
High cohesion internally
+
Low unnecessary coupling externally
+
Explicit dependencies
+
Strong contracts
```

Part 1 — Single Responsibility Principle

31.6 What Is SRP?

SRP says:

> **A class or module should have one cohesive responsibility and therefore one primary reason to change.**

Microsoft's architecture guidance expresses SRP as an object having one responsibility and one reason to change.

31.7 SRP Is Not “One Method Per Class”

Bad interpretation:

EXAMPLE / FLOW

```
One class
    →
one method
```

SOLID / DESIGN

That would create meaningless fragmentation.

A class may have many methods if they all support the same responsibility.

Example:

C#

```
public sealed class GiftValidator
{
    public void ValidateAmount(
        Gift gift)
    {
    }

    public void ValidateCampaign(
        Gift gift)
    {
    }

    public void ValidateAllocations(
        Gift gift)
    {
    }
}
```

All methods belong to:

EXAMPLE / FLOW

```
Gift validation
```

31.8 Bad SRP Example

C#

```
public sealed class GiftService
{
    public bool Validate(
        Gift gift)
    {
        // Validation.
        return true;
    }

    public void Save(
        Gift gift)
    {
        // SQL.
    }

    public void SendReceipt(
        Gift gift)
    {
        // SMTP.
    }

    public byte[] CreatePdf(
        Gift gift)
    {
```

```
        // PDF.
        return [];
    }

    public void WriteAudit(
        Gift gift)
    {
        // Logging.
    }
}
```

SOLID / DESIGN

Reasons to change include:

EXAMPLE / FLOW

```
Gift validation changes
Database changes
Email provider changes
PDF library changes
Audit requirements change
```

That is many axes of change.

31.9 Better SRP Design

EXAMPLE / FLOW

```
GiftValidator
    →
Validation rules

GiftRepository
    →
Persistence

ReceiptSender
    →
Receipt delivery

ReceiptDocumentGenerator
    →
PDF/document generation

AuditWriter
    →
Audit recording

GiftApplicationService
    →
Orchestrates the use case
```

The orchestrator can coordinate several dependencies without owning each dependency's implementation responsibility.

31.10 SRP Fundraising Example

C#

```
public sealed class GiftApplicationService
{
    private readonly
        IGiftValidator
        _validator;
```

```
    private readonly
        IGiftRepository
        _repository;

    private readonly
        IReceiptSender
        _receiptSender;

    public GiftApplicationService(
        IGiftValidator validator,
        IGiftRepository repository,
        IReceiptSender receiptSender)
    {
        _validator = validator;

        _repository = repository;

        _receiptSender = receiptSender;
    }

    public async Task ProcessAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        _validator.Validate(
            gift);

        await _repository.SaveAsync(
            gift,
            cancellationToken);

        await _receiptSender.SendAsync(
            gift,
            cancellationToken);
    }
}
```

SOLID / DESIGN

Responsibility:

EXAMPLE / FLOW

```
Orchestrate the Process Gift use case
```

Not:

EXAMPLE / FLOW

```
Implement SQL
Implement SMTP
Implement validation
```

31.11 Constructor Dependency Smell

Suppose:

C#

```
public GiftService(
    IRepository repository,
    ILogger logger,
    IEmailSender email,
    IPdfGenerator pdf,
    IPaymentGateway payment,
    ICache cache,
    IAuditWriter audit,
```

```
    IMapper mapper,
    IMessageBus bus,
    IClock clock,
    IFileStore fileStore)
```

SOLID / DESIGN

The number itself is not automatically wrong.

But it is a strong question:

EXAMPLE / FLOW

```
Does this class own too many responsibilities?
```

Microsoft's DI guidance explicitly says many constructor dependencies can indicate an SRP violation and suggests splitting responsibilities.

31.12 When Not to Split Further

Bad overdesign:

EXAMPLE / FLOW

```
GiftAmountReader
GiftAmountChecker
GiftAmountComparer
GiftAmountValidationInvoker
GiftAmountValidationCoordinator
```

for one simple rule.

SRP does not mean:

EXAMPLE / FLOW

```
maximum number of classes
```

It means:

EXAMPLE / FLOW

```
cohesive responsibilities
```

Part 2 — Open/Closed Principle

31.13 What Is OCP?

OCP says:

> **Software entities should be open for extension but closed for modification.**

Meaning:

EXAMPLE / FLOW

```
New supported behavior
    ↓
ideally add new implementation

instead of

editing a central class every time
```

The principle is about creating stable variation points, often through polymorphism, composition, configuration, delegates, or strategies.

SOLID / DESIGN

31.14 Bad OCP Example

C#

```
public decimal CalculateFee(
    Gift gift)
{
    if (
        gift.PaymentMethod
        ==
        PaymentMethod.CreditCard)
    {
        return
            gift.TotalAmount
            *
            0.03m;
    }

    if (
        gift.PaymentMethod
        ==
        PaymentMethod.Ach)
    {
        return
            1.00m;
    }

    if (
        gift.PaymentMethod
        ==
        PaymentMethod.Check)
    {
        return
            0m;
    }

    throw new
        NotSupportedException();
}
```

Now add:

EXAMPLE / FLOW

```
PayPal
Apple Pay
Wire Transfer
Crypto
```

Every new type requires modifying the same method.

31.15 OCP with Strategy

C#

```
public interface
    IPaymentFeeCalculator
{
    bool CanHandle(
        PaymentMethod method);
```

```
    decimal Calculate(
        Gift gift);
}
```

SOLID / DESIGN

Credit card:

C#

```
public sealed class
    CreditCardFeeCalculator
    :
    IPaymentFeeCalculator
{
    public bool CanHandle(
        PaymentMethod method)
        =>
        method
        == PaymentMethod.CreditCard;

    public decimal Calculate(
        Gift gift)
        =>
        gift.TotalAmount
        *
        0.03m;
}
```

ACH:

C#

```
public sealed class
    AchFeeCalculator
    :
    IPaymentFeeCalculator
{
    public bool CanHandle(
        PaymentMethod method)
        =>
        method
        == PaymentMethod.Ach;

    public decimal Calculate(
        Gift gift)
        =>
        1m;
}
```

New payment type:

EXAMPLE / FLOW

```
add implementation
```

instead of modifying the existing calculators.

31.16 OCP Does Not Mean “Never Modify Code”

Requirements change.

Bug fixes happen.

OCP does not literally prohibit modification.

The goal is:

Common variation should happen through intended extension points rather than repeatedly editing stable central logic.

31.17 Do Not Abstract Before You Know the Variation

Overengineering:

EXAMPLE / FLOW

```
One receipt format exists

but create:
IReceiptStrategyFactory
IReceiptStrategyResolver
IReceiptPipelineProvider
IReceiptContextBuilder
```

because:

EXAMPLE / FLOW

```
"Maybe someday..."
```

That violates another principle:

EXAMPLE / FLOW

```
YAGNI
```

Create extension points when variation is real or reasonably imminent.

Part 3 — Liskov Substitution Principle

31.18 What Is LSP?

LSP says:

An implementation of a subtype or abstraction must be safely usable wherever the base contract is expected.

This is stronger than:

EXAMPLE / FLOW

```
Compiler allows assignment.
```

A subtype can compile correctly and still violate behavioral expectations.

Microsoft summarizes LSP as requiring a subtype to be safely usable wherever the parent type is expected.

31.19 LSP Is About Behavior

Suppose:

C#

```
public interface IReceiptSender
{
    Task SendAsync(
```

```
        Gift gift,
        CancellationToken cancellationToken);
}
```


Implementation A:

C#

```
public sealed class
    EmailReceiptSender
    :
    IReceiptSender
{
    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        // Send.
        return Task.CompletedTask;
    }
}
```

Implementation B:

C#

```
public sealed class
    FakeReceiptSender
    :
    IReceiptSender
{
    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        throw new
            NotSupportedException();
    }
}
```

If callers reasonably expect:

EXAMPLE / FLOW

```
Every IReceiptSender can send receipts
```

then the second implementation violates the contract.

31.20 Contract Components

LSP asks implementations to respect things such as:

EXAMPLE / FLOW

```
Preconditions
Postconditions
Invariants
Expected exceptions
State changes
Return semantics
Side-effect expectations
```

31.21 Preconditions

Base contract:

EXAMPLE / FLOW

```
Accept any non-null Gift.
```

Derived implementation:

EXAMPLE / FLOW

```
Accept only Gift.Amount > $10,000.
```

The derived type has strengthened the requirement.

That may violate substitutability.

31.22 Postconditions

Suppose contract says:

EXAMPLE / FLOW

```
On successful SaveAsync,
Gift has been persisted.
```

An implementation that silently ignores the save violates that promise.

31.23 Exceptions and LSP

Suppose callers understand:

EXAMPLE / FLOW

```
duplicate external reference
    →
DuplicateGiftException
```

but one implementation unexpectedly throws:

EXAMPLE / FLOW

```
NotSupportedException
```

for ordinary valid usage.

That can violate behavioral substitutability.

The point is not:

EXAMPLE / FLOW

```
all implementations throw identical exceptions
```

but:

EXAMPLE / FLOW

```
implementation honors the abstraction's intended contract
```

31.24 Classic Shape Problem

Imagine:

```C#
class Rectangle
{
    public virtual int Width
    {
        get;
        set;
    }

    public virtual int Height
    {
        get;
        set;
    }
}
```

Then:

```C#
class Square :
    Rectangle
```

forces width and height to remain equal.

Code written for `Rectangle` may expect:

```C#
rectangle.Width = 5;

rectangle.Height = 10;
```

and assume:

EXAMPLE / FLOW

```
Width = 5
Height = 10
```

A `Square` cannot preserve both its own invariant and that Rectangle behavior.

Inheritance may therefore be the wrong relationship.

31.25 LSP Design Lesson

Ask:

> **Does every implementation truly satisfy the semantic contract?**

Not merely:

EXAMPLE / FLOW

```
Can I make the compiler accept it?
```

Part 4 — Interface Segregation Principle

31.26 What Is ISP?

ISP says:

Clients should not be forced to depend on operations they do not need.

SOLID / DESIGN

Large “everything” interfaces cause unnecessary coupling.

31.27 Bad Interface

C#

```
public interface IGiftManager
{
    Gift Get(
        long id);

    void Save(
        Gift gift);

    void Delete(
        long id);

    void SendReceipt(
        long id);

    byte[] ExportPdf(
        long id);

    void RebuildSearchIndex();

    void ClearCache();

    void RunAnnualReport();
}
```

A read-only endpoint needing:

EXAMPLE / FLOW

```
Get
```

now depends conceptually on an interface containing:

EXAMPLE / FLOW

```
Delete
Email
PDF
Cache
Reporting
```

31.28 Better Segregated Interfaces

C#

```
public interface IGiftReader
{
    Task<Gift?> FindAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

```C#
public interface IGiftWriter
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

```C#
public interface IReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

```C#
public interface IGiftExporter
{
    Task<Stream> ExportAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

Each consumer asks for only what it needs.

31.29 ISP Does Not Mean One-Method Interfaces Everywhere

Bad interpretation:

```EXAMPLE / FLOW
Every method
    →
new interface
```

Interfaces should represent cohesive roles.

Example:

```C#
public interface IGiftRepository
{
    Task<Gift?> FindAsync(...);

    Task AddAsync(...);

    Task SaveChangesAsync(...);
}
```

may be perfectly cohesive.

31.30 ISP and Capabilities

A useful approach is thinking in terms of roles:

```EXAMPLE / FLOW
Readable
```

```
Writable
  *
Exportable

Notifiable
```

SOLID / DESIGN

rather than:

EXAMPLE / FLOW

```
One gigantic manager abstraction
```

Part 5 — Dependency Inversion Principle

31.31 What Is DIP?

DIP says broadly:

> **High-level policy should not depend directly on low-level implementation details. Both should depend on abstractions appropriate to the policy boundary.**

Microsoft's architectural guidance describes dependency direction as moving toward abstractions rather than implementation details.

31.32 Bad Dependency Direction

EXAMPLE / FLOW

```
GiftApplicationService
        ↓
SqlConnection
        ↓
SQL Server
```

Code:

C#

```
public sealed class
    GiftApplicationService
{
    public async Task ProcessAsync(
        Gift gift)
    {
        using SqlConnection connection =
            new(
                "...");

        // SQL.
    }
}
```

The high-level application use case knows:

EXAMPLE / FLOW

```
SQL provider
Connection construction
Persistence commands
```

31.33 Inverted Dependency

SOLID / DESIGN

Define:

```
C#
public interface IGiftRepository
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

High-level:

```
C#
public sealed class
    GiftApplicationService
{
    private readonly
        IGiftRepository
        _repository;

    public GiftApplicationService(
        IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

Infrastructure:

```
C#
public sealed class
    SqlGiftRepository
    :
    IGiftRepository
{
    public Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        // EF Core / SQL implementation.
        return Task.CompletedTask;
    }
}
```

Compile-time dependency:

```
EXAMPLE / FLOW
Application
    ↓
IGiftRepository abstraction

Infrastructure
    ↓
implements IGiftRepository
```

31.34 Dependency Injection vs Dependency Inversion

These are related but different.

Dependency Inversion

EXAMPLE / FLOW
```
Design principle
```

It answers:

EXAMPLE / FLOW
```
What should depend on what?
```

Dependency Injection

EXAMPLE / FLOW
```
Construction/composition technique
```

It answers:

EXAMPLE / FLOW
```
How does an object receive dependencies?
```

Example:

C#
```
public GiftService(
    IGiftRepository repository)
```

31.35 Inversion of Control

IoC is broader:

EXAMPLE / FLOW
```
Control of creation/execution
is delegated to framework/container
```

ASP.NET Core DI is one form of IoC.

31.36 ASP.NET Core DI Example

C#
```
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        IReceiptSender,
        EmailReceiptSender>();

builder.Services
    .AddScoped<
        GiftApplicationService>();
```

Then:

C#

```
public GiftController(
    GiftApplicationService service)
{
    _service = service;
}
```

SOLID / DESIGN

The container constructs the object graph.

ASP.NET Core's DI guidance recommends constructor injection, avoiding direct instantiation of dependent classes, and keeping services small and testable.

31.37 Explicit Dependencies

Bad:

C#

```
public sealed class GiftService
{
    public Task ProcessAsync()
    {
        GiftRepository repo =
            ServiceLocator.Get<
                GiftRepository>();

        // ...
    }
}
```

The constructor suggests:

EXAMPLE / FLOW

```
GiftService needs nothing
```

but runtime reality says:

EXAMPLE / FLOW

```
GiftService secretly needs GiftRepository
```

Better:

C#

```
public GiftService(
    IGiftRepository repository)
```

Now dependency requirements are visible.

Microsoft's architectural guidance calls this the Explicit Dependencies Principle.

Part 6 — Separation of Concerns

31.38 What Is Separation of Concerns?

SoC says:

> ⓘ Different kinds of work should be separated into appropriate components/layers.

For example:

EXAMPLE / FLOW

```
API
    →
HTTP concern

Application layer
    →
Use-case orchestration

Domain
    →
Business rules

Infrastructure
    →
Database / external systems
```

SOLID / DESIGN

Microsoft explicitly identifies separation of concerns as a key architectural principle and a motivation for layering.

31.39 Fundraising Layer Example

EXAMPLE / FLOW

```
API
    ↓
GiftController
```

Responsibility:

EXAMPLE / FLOW

```
HTTP input/output
Status codes
Request binding
```

EXAMPLE / FLOW

```
Application
    ↓
ProcessGiftHandler
```

Responsibility:

EXAMPLE / FLOW

```
Orchestrate use case
```

EXAMPLE / FLOW

```
Domain
    ↓
Gift
GiftAllocation
```

Responsibility:

EXAMPLE / FLOW

```
Business invariants
```

EXAMPLE / FLOW

```
Infrastructure
    ↓
GiftRepository
EmailReceiptSender
```

SOLID / DESIGN

Responsibility:

EXAMPLE / FLOW

```
Technical implementation
```

31.40 Bad Controller

C#

```
[HttpPost]
public async Task<IActionResult>
    Create(
        CreateGiftRequest request)
{
    if (
        request.Amount <= 0)
    {
        return BadRequest();
    }

    // Create SqlConnection.
    // Insert Gift SQL.
    // Call payment API.
    // Generate receipt.
    // Send SMTP.
    // Update cache.
    // Publish message.

    return Ok();
}
```

This controller owns:

EXAMPLE / FLOW

```
HTTP
Business logic
SQL
Integration
Messaging
```

31.41 Better Controller

C#

```
[HttpPost]
public async Task<
    ActionResult<GiftResponse>>
    Create(
        CreateGiftRequest request,
        CancellationToken cancellationToken)
{
    GiftResponse response =
        await _service.CreateAsync(
            request,
            cancellationToken);
```

```
    return CreatedAtAction(
        nameof(Get),
        new
        {
            giftId =
                response.GiftId
        },
        response);
}
```

Controller responsibility stays focused.

Part 7 — Encapsulation and Information Hiding

31.42 Encapsulation

Encapsulation means protecting internal state behind well-defined behavior.

Bad:

C#

```
public sealed class Gift
{
    public decimal Amount;

    public List<GiftAllocation>
        Allocations = [];
}
```

Anyone can create invalid state.

Better

C#

```
public sealed class Gift
{
    private readonly
        List<GiftAllocation>
        _allocations = [];

    public decimal Amount
    {
        get;
        private set;
    }

    public IReadOnlyList<GiftAllocation>
        Allocations
        =>
        _allocations;

    public void AddAllocation(
        GiftAllocation allocation)
    {
        if (
            allocation.Amount <= 0)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(allocation));
```

```
        }

        decimal current =
            _allocations.Sum(
                x =>
                    x.Amount);

        if (
            current
            +
            allocation.Amount
            >
            Amount)
        {
            throw new
                InvalidOperationException(
                    "Allocation exceeds gift amount.");
        }

        _allocations.Add(
            allocation);
    }
}
```

SOLID / DESIGN

Outside code cannot bypass the invariant through normal API usage.

Microsoft's architecture guidance connects encapsulation with insulating internal implementation and reducing coupling.

31.43 Information Hiding

Encapsulation is not merely:

```
EXAMPLE / FLOW
make fields private
```

It is also:

```
EXAMPLE / FLOW
Hide implementation decisions
that callers should not depend on.
```

Example:

```
C#
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

Consumer does not need to know whether storage is:

```
EXAMPLE / FLOW
EF Core
Dapper
SQL
In-memory test implementation
```

Part 8 — DRY

SOLID / DESIGN

31.44 What Is DRY?

DRY means:

EXAMPLE / FLOW

```
Don't Repeat Yourself
```

More precisely:

> **A business rule or knowledge concept should not have multiple independent authoritative implementations.**

Microsoft's architecture guidance describes duplicated business behavior as dangerous because future requirement changes can leave copies inconsistent.

31.45 Bad Duplication

Controller:

C#

```
if (
    request.Amount <= 0
    ||
    request.Amount
    >
    1_000_000m)
{
}
```

Service:

C#

```
if (
    amount <= 0
    ||
    amount
    >
    1_000_000m)
{
}
```

Background import:

C#

```
if (
    gift.Amount <= 0
    ||
    gift.Amount
    >
    1_000_000m)
{
}
```

Now change maximum to:

EXAMPLE / FLOW

```
2,000,000
```

SOLID / DESIGN

Someone updates two places but misses one.

31.46 Better

C#

```
public sealed class GiftAmount
{
    public const decimal
        Maximum = 2_000_000m;

    public static void Validate(
        decimal amount)
    {
        if (
            amount
            is <= 0m
            or
            >
            Maximum)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(amount));
        }
    }
}
```

One authoritative rule.

31.47 DRY Does Not Mean “Never Repeat Code”

Two blocks that happen to look similar may represent different business concepts.

Prematurely merging them can create artificial coupling.

Ask:

EXAMPLE / FLOW

```
Are these copies of the same knowledge?
```

not merely:

EXAMPLE / FLOW

```
Do these five lines look similar?
```

Part 9 — KISS

31.48 What Is KISS?

KISS means:

EXAMPLE / FLOW

```
Keep It Simple
```

A design should be as simple as possible while still correctly satisfying known requirements.

Example:

Need two receipt implementations?

SOLID / DESIGN

Simple:

```
C#
IReceiptSender
```

plus:

```
EXAMPLE / FLOW
EmailReceiptSender
PrintReceiptSender
```

Potentially excessive:

```
EXAMPLE / FLOW
ReceiptAbstractFactory
ReceiptFactoryProvider
ReceiptFactoryResolver
ReceiptStrategyRegistry
ReceiptPipelineBuilder
ReceiptExecutionContextFactory
```

unless those abstractions solve real complexity.

31.49 Simple Does Not Mean Primitive

A simple architecture can still have:

```
EXAMPLE / FLOW
Controllers
Application services
Domain objects
Repositories
Interfaces
DI
```

if those elements correspond to real concerns.

KISS means:

```
EXAMPLE / FLOW
No unnecessary complexity
```

not:

```
EXAMPLE / FLOW
Everything in Program.cs
```

Part 10 — YAGNI

31.50 What Is YAGNI?

YAGNI means:

Do not implement speculative flexibility without a real requirement.

Example:

Current requirement:

```
EXAMPLE / FLOW
Only SQL Server persistence.
```

You probably do not need to immediately build support for:

```
EXAMPLE / FLOW
MongoDB
Cosmos DB
Redis primary persistence
PostgreSQL
Flat files
Oracle
```

unless architecture requirements actually demand it.

31.51 YAGNI and OCP Balance

OCP says:

```
EXAMPLE / FLOW
Design extension points for real variation.
```

YAGNI says:

```
EXAMPLE / FLOW
Do not invent speculative variation.
```

Healthy balance:

```
EXAMPLE / FLOW
Known changing dimension?
    →
abstract intentionally

Unknown hypothetical future?
    →
keep design simple
```

Part 11 — Composition over Inheritance

31.52 Why Prefer Composition Often?

Inheritance creates:

```
EXAMPLE / FLOW
is-a
```

relationships.

Composition creates:

EXAMPLE / FLOW

```
has-a / uses-a
```

relationships.

Example inheritance:

C#

```
class GiftService :
    EmailService
```

This probably makes little conceptual sense.

Better:

C#

```
class GiftService
{
    private readonly
        IEmailSender
        _emailSender;
}
```

Now:

EXAMPLE / FLOW

```
GiftService
uses EmailSender
```

31.53 Why Composition Is Flexible

Inheritance binds behavior into a type hierarchy.

Composition allows behavior to be replaced independently:

EXAMPLE / FLOW

```
GiftService
    ↓
IReceiptSender
    ├─ EmailReceiptSender
    ├─ PrintReceiptSender
    └─ TestReceiptSender
```

No GiftService inheritance hierarchy is required.

31.54 When Inheritance Is Good

Inheritance still fits when:

EXAMPLE / FLOW

```
True substitutable is-a relationship exists
Base contract is stable
LSP can be honored
Shared polymorphic behavior is meaningful
```

Do not interpret:

EXAMPLE / FLOW

```
Composition over inheritance
```

SOLID / DESIGN

as:

EXAMPLE / FLOW

```
Inheritance is forbidden
```

Part 12 — Law of Demeter

31.55 Law of Demeter

A practical version says:

> **An object should collaborate mainly with its direct dependencies rather than navigating deeply through another object's internals.**

Problem:

C#

```
string code =
    gift
        .Campaign
        .Owner
        .Department
        .Address
        .Country
        .Code;
```

The caller knows a huge amount about the internal object graph.

31.56 Why Deep Navigation Is Fragile

If structure changes:

EXAMPLE / FLOW

```
Campaign.Owner
    →
Campaign.Team.Owner
```

many callers break.

Alternative:

C#

```
string GetCampaignCountryCode()
```

or better:

EXAMPLE / FLOW

```
appropriate domain/query abstraction
```

depending on the context.

Do not mechanically create forwarding methods for every property. The principle is about reducing unnecessary structural knowledge.

Part 13 — Tell, Don't Ask

SOLID / DESIGN

31.57 Tell, Don't Ask

Weak domain behavior:

```C#
if (
    gift.Status
    ==
    GiftStatus.Pending)
{
    gift.Status = GiftStatus.Completed;
}
```

Caller asks for internal state, decides rule, mutates state.

Better:

```C#
gift.Complete();
```

Inside:

```C#
public void Complete()
{
    if (
        Status
        !=
        GiftStatus.Pending)
    {
        throw new
            InvalidOperationException();
    }

    Status = GiftStatus.Completed;
}
```

The object owns its state transition rule.

Part 14 — Principle of Least Knowledge and Explicit APIs

A public API should expose:

EXAMPLE / FLOW

```
what callers need
```

not:

EXAMPLE / FLOW

```
everything implementation knows
```

Instead of:

```C#
public DbContext Context
{
```

```
    get;
}
```

SOLID / DESIGN

from your domain service, expose the business operation.

This reduces accidental coupling.

Part 15 — SOLID Applied Together

Consider a receipt workflow.

Bad:

C#

```
public sealed class GiftService
{
    public async Task CompleteAsync(
        Gift gift)
    {
        if (
            gift.Amount <= 0)
        {
            throw new Exception();
        }

        // EF SQL logic

        if (
            gift.ReceiptType == "Email")
        {
            // SMTP implementation
        }
        else if (
            gift.ReceiptType == "Print")
        {
            // PDF implementation
        }

        // audit implementation
    }
}
```

Problems:

EXAMPLE / FLOW

```
SRP
    →
validation + persistence + receipt + audit

OCP
    →
new receipt type modifies GiftService

ISP
    →
likely manager abstractions become broad

DIP
    →
depends directly on implementation details

LSP
    →
```

```
future polymorphic receipt implementations
may not honor consistent contract
```

31.58 Improved Design

C#

```
public interface IGiftRepository
{
    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

C#

```
public interface IReceiptSender
{
    bool CanHandle(
        ReceiptType type);

    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

C#

```
public interface IAuditWriter
{
    Task WriteGiftCompletedAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Orchestrator:

C#

```
public sealed class
    CompleteGiftService
{
    private readonly
        IGiftRepository
        _repository;

    private readonly
        IReadOnlyList<IReceiptSender>
        _receiptSenders;

    private readonly
        IAuditWriter
        _auditWriter;

    public CompleteGiftService(
        IGiftRepository repository,
        IEnumerable<IReceiptSender>
            receiptSenders,
        IAuditWriter auditWriter)
    {
        _repository = repository;

        _receiptSenders = receiptSenders.ToList();
```

```
        _auditWriter = auditWriter;
    }

    public async Task CompleteAsync(
        Gift gift,
        ReceiptType receiptType,
        CancellationToken cancellationToken)
    {
        gift.Complete();

        await _repository.SaveAsync(
            gift,
            cancellationToken);

        IReceiptSender sender =
            _receiptSenders
                .Single(
                    sender =>
                        sender.CanHandle(
                            receiptType));

        await sender.SendAsync(
            gift,
            cancellationToken);

        await _auditWriter
            .WriteGiftCompletedAsync(
                gift,
                cancellationToken);
    }
}
```

SOLID / DESIGN

31.59 How Principles Apply

SRP:

EXAMPLE / FLOW

```
CompleteGiftService
    →
orchestrates completion
```

OCP:

EXAMPLE / FLOW

```
New ReceiptSender implementation
    →
can extend behavior
```

LSP:

EXAMPLE / FLOW

```
Every IReceiptSender
must satisfy receipt contract
```

ISP:

EXAMPLE / FLOW

```
Receipt sender doesn't also expose
repository/report/cache methods
```

DIP:

EXAMPLE / FLOW

```
Application depends on abstractions
not SMTP/EF implementation classes
```

SOLID / DESIGN

31.60 SOLID and Testing

Tightly coupled:

C#

```
public sealed class GiftService
{
    private readonly
        SqlGiftRepository
        _repository = new();

    private readonly
        EmailReceiptSender
        _sender = new();
}
```

Unit test may require real infrastructure.

Testable

C#

```
public GiftService(
    IGiftRepository repository,
    IReceiptSender sender)
```

Test:

C#

```
IGiftRepository repository = new FakeGiftRepository();

IReceiptSender sender = new FakeReceiptSender();

GiftService service =
    new(
        repository,
        sender);
```

Now the test controls dependencies.

31.61 Do Not Create Interfaces Only for Mocking

An interface is most valuable when it expresses:

EXAMPLE / FLOW

```
A meaningful abstraction
A replaceable boundary
A role/capability
```

Bad:

EXAMPLE / FLOW

```
IGiftDto
IGiftMapperHelper
IGiftNameFormatterInternal
```

for every trivial class solely because:

EXAMPLE / FLOW

```
"SOLID says use interfaces."
```

SOLID does not say every class needs an interface.

31.62 SOLID and Performance

More abstraction can introduce:

EXAMPLE / FLOW

```
More objects
More indirection
More virtual/interface calls
More allocations in poorly designed pipelines
```

But in ordinary ASP.NET Core applications, architecture mistakes such as:

EXAMPLE / FLOW

```
bad SQL
N+1 queries
unbounded allocations
network latency
serialization
```

normally dominate a simple interface call.

Do not sacrifice maintainability for hypothetical nanoseconds without measurement.

31.63 SOLID and Memory

Overengineering can create excessively large object graphs:

EXAMPLE / FLOW

```
Controller
  ↓
Service
  ↓
Coordinator
  ↓
Factory
  ↓
Resolver
  ↓
Provider
  ↓
Adapter
  ↓
Wrapper
```

Each dependency may be small, but complexity itself has cost.

Prefer meaningful abstractions.

31.64 SOLID and Thread Safety

SOLID / DESIGN

Dependency inversion does not make an implementation thread-safe.

Example:

```C#
public interface IGiftCache
{
}
```

implemented by:

```C#
SingletonGiftCache
```

with a mutable:

```C#
Dictionary<long, Gift>
```

Still needs concurrency-safe design.

Abstraction:

EXAMPLE / FLOW

```
does not remove runtime behavior requirements
```

31.65 SOLID and Security

Bad abstraction can hide authorization.

Example:

```C#
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long giftId);
}
```

In a multi-tenant application, a caller may accidentally request another tenant's Gift.

Potentially stronger contract:

```C#
Task<Gift?> FindAsync(
    long tenantId,
    long giftId,
    CancellationToken cancellationToken);
```

or enforce tenant scope at an appropriate architectural boundary.

Design principles should make security requirements:

EXAMPLE / FLOW

```
explicit
```

not hidden.

31.66 Service Locator Anti-Pattern

Bad:

```C#
public sealed class GiftService
{
    public void Process()
    {
        IGiftRepository repo =
            ServiceLocator
                .Resolve<
                    IGiftRepository>();
    }
}
```

Dependencies are hidden.

Constructor injection:

```C#
public GiftService(
    IGiftRepository repository)
```

makes them visible.

31.67 Static Dependency Anti-Pattern

Bad:

```C#
public static class GiftHelpers
{
    public static EmailSender
        Email = new();
}
```

Now tests and consumers share global mutable state.

Static utility methods can be fine when:

```EXAMPLE / FLOW
Pure
Stateless
Dependency-free
```

but not as a substitute for proper service lifetime/composition.

31.68 God Class

Signs:

```EXAMPLE / FLOW
2,000+ lines
20 dependencies
dozens of unrelated methods
database + messaging + validation + reporting
changes for almost every feature
```

A God class often violates:

EXAMPLE / FLOW

```
SRP
SoC
DIP
ISP
```

Refactor by identifying cohesive responsibilities.

31.69 Anemic Over-Splitting

Opposite problem:

EXAMPLE / FLOW

```
GiftAmountGetter
GiftAmountSetter
GiftAmountRuleProvider
GiftAmountCheckExecutor
GiftAmountComparisonStrategy
```

for:

EXAMPLE / FLOW

```
if amount <= 0
```

That creates accidental complexity.

Good design is not measured by:

EXAMPLE / FLOW

```
number of interfaces
```

31.70 Common Mistake — SRP Means One Method

False.

SRP concerns reasons to change and responsibility cohesion.

31.71 Common Mistake — OCP Means Never Edit Existing Code

False.

It means known variation should preferably be handled through stable extension mechanisms rather than repeatedly modifying central logic.

31.72 Common Mistake — LSP Is Just Type Compatibility

False.

This compiles:

C#

```
IReceiptSender sender = new BrokenReceiptSender();
```

but if the implementation violates the behavioral contract, substitution is not valid.

31.73 Common Mistake — ISP Means Tiny Interfaces Everywhere

SOLID / DESIGN

False.

An interface can contain several methods when they represent one cohesive role.

31.74 Common Mistake — DIP Means “Use DI Container”

False.

A container is a mechanism.

The principle concerns dependency direction.

You can follow DIP manually without a container.

You can also use a DI container while still designing terrible abstractions.

31.75 Common Mistake — DRY Everything

Two identical-looking lines may represent different concepts.

Do not merge unrelated business rules just to eliminate visual duplication.

31.76 Common Mistake — Premature OCP

Do not create strategy systems for variation that does not exist.

Apply YAGNI.

31.77 Common Mistake — Inheritance for Code Reuse

Bad reason:

EXAMPLE / FLOW

```
"I want these three methods,
so I will inherit."
```

Inheritance is a semantic relationship, not merely a code-copy reduction mechanism.

Prefer composition if:

EXAMPLE / FLOW

```
uses-a
```

describes the relationship better than:

EXAMPLE / FLOW

```
is-a
```

31.78 Common Mistake — Repository for Every Object Automatically

Architecture principles do not require:

EXAMPLE / FLOW

```
Repository for every table
Service for every repository
Interface for every service
```

SOLID / DESIGN

Abstractions should serve actual architectural boundaries.

31.79 Common Mistake — Business Rules in Controller

Controllers should generally coordinate HTTP concerns, not become the authoritative home of domain rules.

If:

EXAMPLE / FLOW

```
Gift amount must be positive
```

is a business invariant, put that rule where all entry points honor it.

31.80 Common Mistake — Business Rules in Infrastructure

Bad:

EXAMPLE / FLOW

```
GiftRepository.Save()
    ↓
quietly decides
whether Gift can complete
```

Persistence infrastructure should not unexpectedly own core business policy.

31.81 Common Mistake — Too Many Interfaces

If:

EXAMPLE / FLOW

```
interface only has exactly one implementation
no likely variation
no architectural boundary
no role abstraction
```

ask whether the interface adds real value.

31.82 Best Practices

Use SRP to keep classes cohesive.

Use OCP where variation is real.

Use LSP to reason about behavior, not merely inheritance syntax.

Use ISP to expose focused client-facing roles.

Use DIP to point compile-time dependencies toward abstractions.

Use constructor injection to make dependencies explicit.

Keep domain logic separate from infrastructure concerns.

Use encapsulation to protect invariants.

Use DRY for duplicated knowledge, not merely duplicated text.

Use KISS to resist needless architecture.

Use YAGNI to resist hypothetical requirements.

SOLID / DESIGN

Prefer composition when inheritance does not represent a genuine substitutable relationship.

Design abstractions around behavior/capabilities.

Do not let SOLID become an excuse for creating dozens of meaningless layers.

31.83 Practical Exercise 1 — SRP

Bad:

C#

```
public sealed class GiftManager
{
    public void Validate()
    {
    }

    public void Save()
    {
    }

    public void Email()
    {
    }

    public void Export()
    {
    }
}
```

Refactor conceptually into:

EXAMPLE / FLOW

```
GiftValidator
GiftRepository
ReceiptSender
GiftExporter
```

and one application service coordinating the use case.

31.84 Practical Exercise 2 — OCP

Current:

C#

```
switch (
    receiptType)
{
    case ReceiptType.Email:
        // ...
        break;

    case ReceiptType.Print:
        // ...
        break;
}
```

Refactor toward:

implementations.

Add a new sender without editing every existing receipt implementation.

31.85 Practical Exercise 3 — LSP

Interface:

```C#
public interface IGiftExporter
{
    Stream Export(
        Gift gift);
}
```

Bad implementation:

```C#
public Stream Export(
    Gift gift)
{
    throw new
        NotSupportedException();
}
```

Question:

Why suspicious?

Answer:

EXAMPLE / FLOW

```
The implementation claims the exporter role
while refusing its primary operation.
```

31.86 Practical Exercise 4 — ISP

Bad:

```C#
public interface IDataService
{
    Gift GetGift();
    void SaveGift();
    void SendReceipt();
    void Export();
    void ClearCache();
}
```

Split by cohesive capabilities.

31.87 Practical Exercise 5 — DIP

Bad:

```C#
public class GiftService
{
    private SqlGiftRepository
        _repository = new();
}
```

Better:

```C#
public GiftService(
    IGiftRepository repository)
```

31.88 Practical Exercise 6 — DRY

Find three duplicated validation rules.

Ask:

EXAMPLE / FLOW

```
Same business knowledge?
```

If yes, centralize.

If merely visually similar, do not merge automatically.

31.89 Practical Exercise 7 — Encapsulation

Bad:

```C#
gift.Status = GiftStatus.Completed;
```

Better:

```C#
gift.Complete();
```

where `Complete()` checks valid transitions.

31.90 Practical Exercise 8 — Composition

Bad relationship:

```C#
class GiftService :
    EmailSender
```

Better:

```C#
class GiftService
{
    private readonly
        IEmailSender
        _emailSender;
}
```

31.91 Practical Exercise 9 — Explicit Dependencies

Bad:

```
C#
GiftRepository.Current.Save(
    gift);
```

Better:

```
C#
public GiftService(
    IGiftRepository repository)
```

31.92 Practical Exercise 10 — KISS

Requirement:

```
EXAMPLE / FLOW
Only one receipt channel today
```

Start with:

```
EXAMPLE / FLOW
One simple implementation
```

Do not build a plugin system unless variation exists or is clearly required.

31.93 Practical Exercise 11 — Multi-Tenant Security

Bad repository API:

```
C#
FindAsync(
    giftId)
```

Ask whether tenant scoping is guaranteed elsewhere.

If not, a safer abstraction may require:

```
C#
FindAsync(
    tenantId,
    giftId,
    cancellationToken)
```

31.94 Practical Exercise 12 — Refactor Large Service

Given a service with:

```
EXAMPLE / FLOW
Validation
SQL
Email
PDF
HTTP payment
```

```
Audit
Cache
```

SOLID / DESIGN

Identify:

EXAMPLE / FLOW

```
Business responsibility
Application orchestration
Infrastructure dependencies
Cross-cutting concerns
```

before creating new classes.

Topic 31 — 25 Interview Questions

1. What is SOLID?

SOLID is a group of five object-oriented design principles:

EXAMPLE / FLOW

```
Single Responsibility
Open/Closed
Liskov Substitution
Interface Segregation
Dependency Inversion
```

They aim to reduce unnecessary coupling and improve maintainability, extensibility, and testability. Microsoft includes these five principles in its current .NET architecture guidance.

2. What is the Single Responsibility Principle?

SRP says a class/module should have one cohesive responsibility and one primary reason to change.

It does not mean one method per class.

A validator can have many validation methods while still having one cohesive responsibility.

3. How can constructor dependencies reveal an SRP problem?

A class requiring many unrelated dependencies may be coordinating too many responsibilities.

For example:

EXAMPLE / FLOW

```
Repository
SMTP
PDF
Payment
Cache
Audit
File system
Message broker
```

in one service should trigger a design review.

Microsoft explicitly notes this as an SRP smell in DI guidance.

4. What is the Open/Closed Principle?

OCP says software should be open to extension while stable existing code is relatively closed to modification.

For example, adding a new `IReceiptSender` implementation should ideally not require rewriting the application's existing receipt senders.

5. Does OCP mean existing code must never change?

No.

Bug fixes and requirement changes require modification.

The principle says known variation should be designed through extension points so that every new variant doesn't repeatedly require invasive modification of stable central logic.

6. What is the Liskov Substitution Principle?

LSP says a subtype or implementation must be safely substitutable for the abstraction/base type expected by callers.

It concerns behavioral contract, not only compiler-compatible inheritance.

7. What kinds of behavior matter for LSP?

Examples include:

EXAMPLE / FLOW

```
Accepted inputs
Returned outputs
State invariants
Expected side effects
Error semantics
Postconditions
```

An implementation that technically implements an interface but refuses valid contract operations can violate LSP.

8. Why is the Rectangle/Square example associated with LSP?

A mutable `Rectangle` contract may allow width and height to vary independently.

A `Square` requires them to remain equal.

Substituting Square can therefore violate behavior expected from Rectangle.

The problem is the contract, not whether inheritance syntax compiles.

9. What is Interface Segregation?

ISP says clients should not depend on operations they do not need.

Prefer focused role interfaces over large “manager” interfaces containing unrelated capabilities.

10. Does ISP require single-method interfaces?

No.

An interface can contain multiple methods when they form one cohesive role.

For example, a repository may reasonably expose several persistence operations.

11. What is Dependency Inversion?

DIP says high-level policy should depend on abstractions rather than low-level implementation details, and implementation details should conform to those abstractions.

12. What is the difference between DIP and dependency injection?

SOLID / DESIGN

DIP is a design principle.

DI is a technique used to supply dependencies to objects.

Constructor injection:

```
C#
public GiftService(
    IGiftRepository repository)
```

is one common mechanism for implementing a DIP-friendly design.

13. What is inversion of control?

IoC means control over creation or execution is handed to an external framework/container rather than the object controlling everything itself.

ASP.NET Core's DI container is an example.

14. Why is constructor injection usually preferred?

It makes required dependencies explicit.

A class cannot be constructed without the collaborators it needs, which improves:

EXAMPLE / FLOW

```
Discoverability
Testability
Validity
Transparency
```

Microsoft's DI guidance recommends constructor injection and avoiding direct instantiation of dependencies.

15. What is separation of concerns?

SoC means different kinds of work should reside in different components/layers.

Typical ASP.NET Core separation:

EXAMPLE / FLOW

```
Controller → HTTP
Application service → use case
Domain → business rules
Infrastructure → database/external systems
```

16. What is encapsulation?

Encapsulation protects internal state and exposes behavior through controlled APIs.

A `Gift` should ideally maintain its own invariants instead of exposing mutable state that any caller can modify arbitrarily.

17. What is DRY?

DRY means avoiding multiple independent implementations of the same knowledge/business rule.

It is not simply eliminating every visually duplicated line of code.

18. What is KISS?

KISS means keeping the design as simple as possible while correctly satisfying real requirements.

It protects against unnecessary factories, strategies, wrappers, abstractions, and layers.

19. What is YAGNI?

YAGNI means:

EXAMPLE / FLOW

```
You Aren't Gonna Need It
```

Do not build speculative features or extension mechanisms for hypothetical future requirements without evidence they are needed.

20. How do OCP and YAGNI coexist?

Use OCP for real or well-established variation.

Use YAGNI to avoid abstracting every hypothetical future possibility.

The balance is:

EXAMPLE / FLOW

```
Known variation → design extension point
Speculative variation → keep simple
```

21. Why is composition often preferred over inheritance?

Composition reduces coupling to a rigid hierarchy and allows behavior to be replaced independently.

Example:

EXAMPLE / FLOW

```
GiftService uses IReceiptSender
```

is often more flexible than:

EXAMPLE / FLOW

```
GiftService inherits ReceiptSender
```

when no genuine is-a relationship exists.

22. What is the Law of Demeter?

It encourages objects to collaborate mainly with direct dependencies rather than navigating deep internal object graphs.

Deep navigation:

C#

```
gift.Campaign.Owner.Department.Address.Country.Code
```

creates structural coupling.

23. Can SOLID hurt a project?

Misapplied SOLID can create overengineering:

EXAMPLE / FLOW

```
Too many interfaces
Too many factories
Too many tiny classes
Hard-to-follow object graphs
Speculative abstractions
```

Principles are decision tools, not mandatory templates.

24. How does SOLID improve testing?

SOLID commonly creates small components with explicit abstractions.

Instead of requiring:

EXAMPLE / FLOW

```
SQL Server
SMTP
HTTP gateway
```

a unit test can provide controlled implementations such as:

EXAMPLE / FLOW

```
Fake repository
Fake sender
Stub clock
```

This isolates the behavior being tested.

25. How would you apply SOLID to the Fundraising Platform?

A practical approach is:

EXAMPLE / FLOW

```
Gift
    →
owns Gift invariants

GiftApplicationService
    →
orchestrates Gift use cases

IGiftRepository
    →
persistence abstraction

GiftRepository
    →
EF Core implementation

IReceiptSender
    →
receipt capability

EmailReceiptSender
PrintReceiptSender
    →
replaceable implementations

IAuditWriter
    →
audit abstraction
```

```
Controller
    →
HTTP boundary
```

Then use:

EXAMPLE / FLOW

```
SRP → cohesive responsibilities
OCP → add new receipt strategies
LSP → all implementations honor contracts
ISP → small capability interfaces
DIP → application depends on abstractions
```

Topic 31 — Quick Revision

Principle	Meaning
SRP	One cohesive responsibility/reason to change
OCP	Extend behavior without repeatedly modifying stable logic
LSP	Implementations preserve behavioral contract
ISP	Clients depend only on capabilities they need
DIP	Dependencies point toward abstractions
DI	Technique for supplying dependencies
IoC	Framework/container controls construction/execution
Cohesion	Related responsibilities belong together
Coupling	Degree of dependency between components
SoC	Separate different kinds of work
Encapsulation	Protect internal state/invariants
Information hiding	Hide implementation detail
DRY	One authority for one piece of knowledge
KISS	Prefer simplest adequate design
YAGNI	Do not build speculative requirements
Composition	Assemble behavior through collaborators
Inheritance	Use for genuine substitutable is-a relationships
Explicit Dependencies	Declare what a type needs
Law of Demeter	Avoid deep dependency navigation
Tell, Don't Ask	Put behavior near owned state

Topic 31 — Complete Fundraising Architecture Example

A clean Gift creation flow might look like:

EXAMPLE / FLOW

```
HTTP POST /gifts
        ↓
GiftController
        ↓
CreateGiftService
        ↓
Gift.Create(...)
        ↓
IGiftRepository
        ↓
GiftRepository / EF Core
        ↓
Commit
        ↓
Receipt / Audit / Outbox orchestration
```

Controller

C#

```
[ApiController]
[Route("api/gifts")]
public sealed class
    GiftController
    :
    ControllerBase
{
    private readonly
        CreateGiftService
        _service;

    public GiftController(
        CreateGiftService service)
    {
        _service = service;
    }

    [HttpPost]
    public async Task<
        ActionResult<CreateGiftResponse>>
        Create(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        CreateGiftResponse response =
            await _service.CreateAsync(
                request,
                cancellationToken);

        return CreatedAtAction(
            nameof(Create),
            new
            {
                giftId =
                    response.GiftId
            },
            response);
    }
}
```

Controller owns:

Domain Gift

C#

```
public sealed class Gift
{
    private readonly
        List<GiftAllocation>
        _allocations = [];

    private Gift(
        long tenantId,
        long constituentId,
        decimal amount)
    {
        if (
            tenantId <= 0)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(tenantId));
        }

        if (
            constituentId <= 0)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(constituentId));
        }

        if (
            amount <= 0m)
        {
            throw new
                ArgumentOutOfRangeException(
                    nameof(amount));
        }

        TenantId = tenantId;

        ConstituentId = constituentId;

        Amount = amount;

        Status = GiftStatus.Pending;
    }

    public long GiftId
    {
        get;
        private set;
    }

    public long TenantId
    {
        get;
    }

    public long ConstituentId
    {
```

```
        get;
    }

    public decimal Amount
    {
        get;
    }

    public GiftStatus Status
    {
        get;
        private set;
    }

    public IReadOnlyList<
        GiftAllocation>
        Allocations
        =>
        _allocations;

    public static Gift Create(
        long tenantId,
        long constituentId,
        decimal amount)
        =>
        new(
            tenantId,
            constituentId,
            amount);

    public void Complete()
    {
        if (
            Status
            !=
            GiftStatus.Pending)
        {
            throw new
                InvalidOperationException(
                    "Only pending gifts can be completed.");
        }

        Status = GiftStatus.Completed;
    }
}
```

SOLID / DESIGN

Gift owns:

EXAMPLE / FLOW

```
Gift invariants
Gift state transition
```

Repository Abstraction

C#

```
public interface IGiftRepository
{
    Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Application Service

SOLID / DESIGN

C#

```
public sealed class
    CreateGiftService
{
    private readonly
        IGiftRepository
        _repository;

    private readonly
        IClock
        _clock;

    public CreateGiftService(
        IGiftRepository repository,
        IClock clock)
    {
        _repository = repository;

        _clock = clock;
    }

    public async Task<
        CreateGiftResponse>
        CreateAsync(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        Gift gift =
            Gift.Create(
                request.TenantId,
                request.ConstituentId,
                request.Amount);

        await _repository.AddAsync(
            gift,
            cancellationToken);

        return new
            CreateGiftResponse(
                gift.GiftId,
                gift.Amount,
                gift.Status,
                _clock.UtcNow);
    }
}
```

Application service owns:

EXAMPLE / FLOW

Create Gift use-case orchestration

Infrastructure Repository

C#

```
public sealed class GiftRepository
    :
    IGiftRepository
{
    private readonly
        FundraisingDbContext
        _dbContext;
```

```
    public GiftRepository(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        await _dbContext.Gifts
            .AddAsync(
                gift,
                cancellationToken);

        await _dbContext
            .SaveChangesAsync(
                cancellationToken);
    }
}
```

Repository implementation owns:

```
EXAMPLE / FLOW
EF Core persistence details
```

Dependency Registration

```
C#
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        CreateGiftService>();
```

Final Architecture Flow

```
EXAMPLE / FLOW
Presentation
    ↓
HTTP Controller
    ↓
Application
    ↓
Use-Case Service
    ↓
Domain
    ↓
Gift / GiftAllocation
```

Compile-time infrastructure direction:

```
EXAMPLE / FLOW
Application/Core
    ↑
Abstraction
```

```
    ↑
Infrastructure implementation
```

Runtime flow may be:

EXAMPLE / FLOW

```
Controller
    ↓
Application Service
    ↓
IGiftRepository
    ↓
GiftRepository
    ↓
SQL Server
```

That is dependency inversion:

EXAMPLE / FLOW

```
Runtime call goes toward infrastructure

Compile-time policy dependency
points toward abstraction
```

Topics 30 + 31 — Combined Mental Model

Topic 30 asks:

EXAMPLE / FLOW

```
How does the runtime execute
and manage my objects?
```

Topic 31 asks:

EXAMPLE / FLOW

```
How should I organize those objects
so the application remains maintainable?
```

Together:

EXAMPLE / FLOW

```
C# Code
    ↓
Well-designed objects/services
    ↓
SOLID + cohesion + encapsulation
    ↓
CLR executes code
    ↓
Objects allocated
    ↓
Managed heap
    ↓
Application references determine lifetime
    ↓
GC reclaims unreachable objects
```

A poorly designed architecture can create poor memory behavior:

EXAMPLE / FLOW

```
Global singleton
    ↓
Huge cache
    ↓
Retains domain graph
    ↓
Objects promoted to Gen 2
    ↓
Memory grows
```

A better design might use:

EXAMPLE / FLOW

```
Explicit lifetime
Bounded cache
Small request-scoped services
Immutable/read-only projections
Streaming
Correct disposal
```

So software design and runtime behavior are connected.

Final Revision Flow

EXAMPLE / FLOW

```
Write business code
    ↓
Use SOLID to organize responsibilities
    ↓
Use DI to compose dependencies
    ↓
Use encapsulation to protect state
    ↓
Avoid unnecessary global state
    ↓
CLR executes methods
    ↓
Managed objects allocated
    ↓
Keep lifetimes appropriate
    ↓
GC handles unreachable objects
    ↓
Measure allocation/memory in production
    ↓
Optimize only proven hot paths
```

The two key lessons are:

> **Topic 30:** Garbage collection frees unreachable managed objects; it cannot fix objects that your architecture accidentally keeps reachable. Good memory behavior therefore depends heavily on lifetime, allocation, caching, pooling, and reference design.

Topic 31: SOLID is not about maximizing the number of interfaces and classes. It is about creating cohesive responsibilities, stable contracts, sensible extension points, substitutable implementations, focused interfaces, and dependency direction that keeps business policy independent of infrastructure details.

SOLID / DESIGN

Next roadmap topic: Topic 32 — Dependency Injection and Service Lifetimes.